Paper Number 173

MEGA — The Maximizing Expected Generalization Algorithm for Learning Complex Query Concepts

Beitao Li and Edward Chang
Electrical & Computer Engineering
University of California, Santa Barbara
echang@ece.ucsb.edu

Abstract

Specifying exact query concepts has become increasingly challenging to end-users. This is because many query concepts (e.g., those for looking up a multimedia object) can be hard to articulate, and articulation can be subjective. In this study, we propose a query-concept learner that learns query criteria through an intelligent sampling process. Our concept learner aims to fulfill two primary design objectives: 1) it has to be expressive in order to model most practical query concepts, and 2) it must learn a concept quickly and with a small number of labeled data since on-line users tend to be too impatient to provide much feedback. To fulfill the first goal, we model query concepts in k-CNF, which can express almost all practical query concepts. To fulfill the second design goal, we propose our *maximizing expected generalization algorithm* (MEGA), which converges to target concepts quickly by its two complementary steps: sample selection and concept refinement. MEGA copes well with moderate noise. Through examples, analysis and experiments, we show that MEGA converges to query concepts significantly faster than traditional methods.

Keywords: query concept, relevance feedback, active learning.

1 Introduction

The query-concept learning approach that we research in this study can be characterized by the following example: Suppose one is asked, "Are the paintings of Leonardo da Vinci more like those of Peter Paul Rubens or those of Raphael?" One is likely to respond with: "What is the basis for the comparison?" Indeed, without knowing the criteria (i.e., the query concept) by which the comparison is to be made, a database system cannot conduct a search. For many search tasks, however, a query concept is hard to articulate, and articulation can be subjective. For instance, in a multimedia search, it is difficult to describe a desired image using low-level attributes such as color, shape, and texture. Different users may perceive the same image differently; and even if an image is perceived similarly, users may use different vocabulary (i.e., different combinations of low-level attributes) to depict it. Furthermore, most users are not trained to specify simple query criteria using for example, SQL. It is thus both necessary (for capturing subjective concepts) and

desirable (for alleviating users from specifying complex queries) to build intelligent search engines that can quickly learn users' query concepts via relevance feedback.

Unfortunately, traditional relevance feedback and learning techniques may not be suitable for online query-concept learning for at least two reasons. First, an online user is typically impatient and cannot be expected to provide a great deal of feedback, but traditional learning methods such as decision trees and neural networks require a large number of training instances to learn a concept [18]. Second, almost all traditional relevance feedback methods [13, 20, 26] require users to provide good examples to seed a query. However, finding good seeds is the job of the search engine itself, and this circular requirement leaves the core problem— learning and understanding users' query concepts—unsolved.

In this study, we propose a query-concept learner that learns a concept through an intelligent sampling process. The design of a query-concept learner must fulfill two primary goals. One, the concept-learner's hypothesis space must not be too restrictive, so it can model most practical query concepts. Two, the concept-learner should grasp a concept quickly and with a small number of labeled instances, since most users do not wait around to provide a great deal of feedback. To fulfill these design goals, we propose the *Maximizing Expected Generalization Algorithm* (MEGA). MEGA models query concepts in k-CNF [14], which can model virtually all practical query concepts[1]. To ensure that target concepts can be learned quickly and with a small number of samples, MEGA employs an intelligent sampling scheme that can quickly gather maximum information for learning users' query concepts. In other words, MEGA maximizes the usefulness of each sample it generates to query the user and therefore requires a far smaller number of samples to learn the target concept than traditional sampling schemes require. MEGA judiciously selects samples at each iteration and uses positive-labeled instances to learn the target concept in k-CNF. At the same time, it uses negative-labeled instances to learn a k-DNF that bounds the uncertain region, in order to select the most informative samples. Our experiments using both synthesized data and real images show that MEGA converges to a target concept much faster than some traditional schemes (e.g., MARS [12, 19, 20]).

1.1 Motivating Example

Below we show an example problem that illustrates the need for MEGA. We will use this simple example throughout this paper to explain various aspects of our approach and to contrast our approach with others. This example models an admission concept that consists of a small number of boolean predicates. In Section 5 we show that MEGA works well to learn complex query concepts of fuzzy predicates for a high-dimensional image dataset.

Suppose Jane plans to apply to a graduate school. Before filling out the forms and paying the application fees, she would like to estimate her chances of being admitted. Since she does not know the admission criteria, she decides to learn the admission concept by induction[2]. She calls

[1] k-CNF is more expressive than k-term DNF, and it has both polynomial sample complexity and time complexity [15, 17].

[2] At first sight, this example may seem very different from, say, an image search scenario, where a user queries similar images by example(s). But if we treat the admission officer as the user who knows what he/she likes and who can, accordingly, label a data as *true* or *false*, and if we treat Jane as the search engine who tries to find out what the admission officer thinks, then it is evident that this example represents a typical search scenario.

up a few friends who applied last year and obtains the information shown in Table 1.

Name	*GPA*	*GRE*	*Has Publications?*	*Was Admitted?*
Joe	*high*	*high*	*false*	*true*
Mary	*high*	*low*	*true*	*true*
Emily	*high*	*low*	*true*	*true*
Lulu	*high*	*high*	*true*	*true*
Anna	*low*	*low*	*true*	*false*
Peter	*low*	*high*	*false*	*false*
Mike	*high*	*low*	*false*	*false*
Pica	*low*	*low*	*false*	*false*

Table 1: Admission Samples.

If we look at the GRE scores in the table, we see that students with either high or low GRE scores were admitted, and both kinds were rejected. Hence, we may conclude that the GRE is irrelevant in the admission process. Likewise, one's publication record does not affect admission acceptance, nor does having a high GPA. It may appear that the admission decision is entirely random. However, the graduate school actually uses a combination of reasonable criteria: it requires a high GPA and either a high GRE or publications. In other words, $Admission : GPA = high \wedge (GRE = high \vee Publications = true)$.

An obvious question arises: "Are all the samples in Table 1 equally useful for learning the target concept?" Apparently not, for several reasons. First, it seems that *Pica*'s record may not be useful since she was unlikely to be admitted (i.e., her record is unlikely to be labeled positive). Second, both Emily and Mary have the same record, so one of these two records is redundant. Third, Lulu's record is perfect and hence does not provide additional insight for learning the admission criteria. This example indicates that choosing samples randomly may not produce useful information for learning a target concept.

In this paper, we will show how and why MEGA can quickly learn a target concept like the example of admission criteria, whereas other methods may not. We also show that our concept learner can tolerate noise, i.e., it works well even when a target concept is not in k-CNF and even when training data contain some errors.

The rest of this paper is organized into six sections. Section 2 discusses related work. Section 3 shows how we model a query concept and how we measure similarity between an object and the query concept. Section 4 depicts the sample selection and concept refinement steps of MEGA. Section 5 presents our experimental results. Finally, we offer our conclusion in Section 6.

2 Related Work

In most inductive learning problems studied in the AI (Artificial Intelligence) community, samples are assumed to be taken randomly in such a way that various statistical properties can be derived conveniently [11, 25]. However, for online applications where the number of samples must be small (lest impatient users be turned away), random sampling is not suitable. Although ensemble techinques such as *bagging* [1], arcing [2], and boosting [23] have been recently proposed to improve

learning accuracy for decision trees and neural networks, these techniques may not help reduce the number of required samples to learn an online query concept.

Relevance feedback techniques proposed by the IR (Information Retrieval) community do perform non-random sampling [13]. The study of [21] puts these query refinement approaches into three categories: *query reweighting*, *query point movement* and *query expansion*.

- *Query reweighting* and *query point movement* [12, 19, 20]. Both query reweighting and query point movement use nearest-neighbor sampling: They return top ranked objects to be marked by the user and refine the query based on the feedback. If the initial query example is good, this nearest-neighbor sampling approach works fine. However, most users may not have a good example to start a query. Refining around bad examples is analogous to trying to find oranges in the middle of an apple orchard by refining one's search to a few rows of apple trees at a time. It will take a long time to find oranges (the desired result).
- *Query expansion* [21, 26]. The *query expansion* approach can be regarded as a multiple-instances sampling approach. The samples of the next round are selected from the neighborhood (not necessarily the nearest ones) of the positive-labeled instances of the previous round. The study of [21] shows that query expansion achieves only a slim margin of improvement (about 10% in precision/recall) over query point movement.

Of late, pool-based learning or active learning has been introduced for choosing good samples from the unlabeled data pool to speed up the learning process. For image retrieval, the PicHunter system [7] uses Bayes' rule to predict the goal image, based upon the users' actions. The system shows that employing active learning can drastically cut down the number of iterations (up to 80% in some experiments). But, the authors also point out that their scheme is computationally intensive since it recomputes conditional probability for all unlabeled samples after each round of user feedback and hence may not scale well with dataset size. MEGA scales well with data dimensionality and dataset size since it retrieves only a small number of informative samples via an effective indexer that we have developed [16].

Another method, the Query by Committee (QBC) algorithm [10], employs a committee of learners to select unlabeled samples that yield maximum information gain. Two major challenges of applying the QBC algorithm to a learning task are 1) finding useful committee members, and 2) finding informative unlabeled instances efficiently. Our proposed MEGA can be regarded as a variant of the QBC algorithm. MEGA provides an effective method for refining committee members (i.e., a k-CNF and a k-DNF hypothesis), and at the same time delimits the boundary of the sampling space for efficiently finding useful samples.

Finally, much traditional work suffers from model bias. Some systems (e.g., [8, 9]) assume that the overall similarity can be expressed as a weighted linear combination of similarities in attributes. Similarly, some systems assume that query concepts are disjunctive [26]. When a query concept does not fit the model assumption, these systems perform poorly. MEGA works well with model bias and moderately noisy feedback.

3 Definitions and Notations

Our learner models query concepts in k-CNF and uses k-DNF to bound the uncertain region for sampling.

Definition 1: **k-CNF**: For constant k, the representation class k-CNF consists of Boolean formulae of the form $c_1 \wedge \cdots \wedge c_\theta$, where each c_i is a disjunction of at most k literals over the Boolean variables $x_1, \ldots, x_n$. No prior bound is placed on θ.

Definition 2: **k-DNF**: For constant k, the representation class k-DNF consists of Boolean formulae of the form $d_1 \vee \cdots \vee d_\theta$, where each d_i is a conjunction of at most k literals over the Boolean variables $x_1, \ldots, x_n$. No prior bound is placed on θ.

In our retrieval system, queries are boolean expressions consisting of predicates connected by the boolean operators $\vee$ (or) and $\wedge$ (and). A predicate on attribute x_k in our system is in the form of P_{x_k}. Note that the membership function [6] of P_{x_k} can be either crispy or fuzzy. A database system consists of a number of predicates. Our goal is to find the proper operators to combine individual predicates to form the query concept. We choose k-CNF to model query concepts, since it can express most practical queries and can be learned via positive-labeled instances in polynomial time [14, 17]. In addition, we use non-positive-labeled instances to reduce our sampling space (bounded by k-DNF), which we will discuss in detail in Section 4.

A k-CNF has three characteristics:

1: The terms (or literals) are combined by the $\wedge$ operator.

2: The predicates in a term are combined by the $\vee$ operator.

3: A term can have at most k predicates.

Suppose we have three predicates P_{x_1}, P_{x_2} and P_{x_3}. The 2-CNF of these predicates is

$$P_{x_1} \wedge P_{x_2} \wedge P_{x_3} \wedge (P_{x_1} \vee P_{x_2}) \wedge (P_{x_1} \vee P_{x_3}) \wedge (P_{x_2} \vee P_{x_3}).$$

Suppose a dataset contains N objects, denoted as O_i, where $i = 1 \ldots N$. Suppose each object can be depicted by M attributes, each of which is denoted by x_k, where $k = 1 \ldots M$. To find objects that best satisfy a k-CNF concept, we need to compute the membership value of an object on the concept. We first compute the membership values at predicate level and then aggregate them into the membership value at concept level. At the predicate level, suppose $F_{x_k}(O_i)$ is the value of object i on attribute x_k. We transform $F_{x_k}(O_i)$ to the predicate level membership value using a membership function μ: $F_{x_k}(O_i) \rightarrow [0, 1]$. Let $P_{x_k}(O_i)$ denote the predicate level membership value of object i for predicate P_{x_k}. $P_{x_k}(O_i) = 1$ means that object i fully satisfies predicate P_{x_k}, $P_{x_k}(O_i) = 0$ means that object i does not satisfy predicate P_{x_k} at all, and $0 < P_{x_k}(O_i) < 1$ means that object i satisfies predicate P_{x_k} to a degree. For example, people's heights can vary from 3 feet to 9 feet. If we want to grasp the fuzzy predicate of "tall", we use a membership function to transform each height value between 3 feet and 9 feet to a membership value between 0 and 1. If we consider people over 7 feet to be definitely tall, then we map heights over 7 feet to 1.0. If we consider people under 5 feet definitely not tall, then we map heights under 5 feet to 0.0. Heights between 5 and 7 feet are considered tall to a degree and

Parameter	*Description*
U	Unlabeled dataset
M	The number of attributes for depicting a data object
N	The number of data objects in U
u	A set of samples selected from the unlabeled set U
x_i	The ith attribute
O_j	The jth object
y_j	The label of the jth object
y	The labeled set u
y^+	The positive-labeled set
y^-	The negative-labeled set
QCS	The set representation of the query concept space in k-CNF
CCS	The set representation of the candidate concept space in k-DNF
d_i	The ith disjunctive term in QCS
c_i	The ith conjunctive term in CCS
t_i	d_i or c_i
$F_{x_k}(O_i)$	The value of O_i on attribute x_k
$P_{x_k}(O_i)$	The membership value of O_i on predicate P_{x_k}
$P_{t_i\|y_j}$	The probability of removing term t_i given y_j
$P_{t_i\|y}$	The probability of removing term t_i given y
K_α	Sample size
K_c	The threshold of eliminating a conjunctive term, c_i
K_d	The threshold of eliminating a disjunctive term, d_i
γ	Voting parameter
$f()$	Func. computing the prob. of removing term t_i given y_j
$Vote()$	Func. computing the aggregated probability of removing t_i
$Sample()$	Sampling func., which selects u from U
$Feedback()$	Labeling function
$Collapsed?()$	The version space has collapsed? true or false
$Converged?()$	The version space has converged? true or false

Table 2: Parameters.

we map them to membership values between 0 and 1.

At the concept level, we use the standard fuzzy rules, as defined by Zadeh [8, 27], to aggregate membership values on individual predicates. To combine M predicate level membership values into one concept level membership value, we need an *M-ary* aggregation function that maps $[0,1]^M$ to $[0,1]$. The rules are as follows:

Conjunctive rule: $P_{x_1 \wedge x_2 \wedge \ldots \wedge x_M}(O_i) = \min\{P_{x_1}(O_i), P_{x_2}(O_i), \ldots P_{x_M}(O_i)\}$.

Disjunctive rule: $P_{x_1 \vee x_2 \vee \ldots \vee x_M}(O_i) = \max\{P_{x_1}(O_i), P_{x_2}(O_i), \ldots P_{x_M}(O_i)\}$.

The objects that best satisfy a concept have the largest aggregated membership values. To assist the reader, Table 2 summarizes the parameters that we have introduced and that we will discuss in the rest of the paper.

4 The MEGA Algorithm

Our learning algorithm consists of two major parts:

- Refinement: Refine the query concept based on positive-labeled instances. The refinement step is carefully designed to tolerate noisy data.

- Sampling: Refine the sampling space based on negative-labeled instances and select samples judiciously for expediting the learning process.

4.1 Refinement

We use Valiant's learning algorithm [24] as the starting point to refine a k-CNF concept. We extend the algorithm to

1: Handle the fuzzy membership functions (described in Section 3),

2: Select samples judiciously to expedite the learning process (Section 4.2), and

3: Tolerate user errors (Section 4.5).

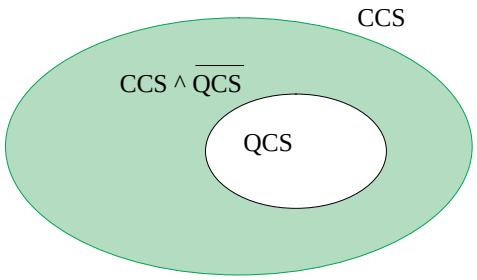


Figure 1: MEGA's Sampling Space: $CCS \wedge \overline{QCS}$.

MEGA initializes the query concept space (QCS) as a k-CNF and the candidate concept space (CCS) as a k-DNF. The QCS starts as the most specific concept and the CCS as the most general concept. The target concept that MEGA learns is more general than the initial QCS and more specific than the initial CCS. It will become evident shortly that MEGA learns the QCS, while at the same time refining the CCS to delimit the boundary of the sampling space. (The shaded area in Figure 1 shows the sampling space between the QCS and the CCS).

Figure 2 depicts the learning algorithm. The first step initializes QCS and CCS. The second step describes how a query concept is refined through relevance feedback. This refinement step consists of the four substeps shown.

Step 2.a: MEGA Generates sample set u from the unlabeled pool U. This step passes QCS, CCS and U to procedure *Sample* to generate K_α samples. We will discuss procedure *Sample* in Section 4.2.

Step 2.b: Mega solicits user feedback. A user marks an object positive if the object fits his/her query concept. An unmarked object is considered negative.

Step 2.c: This is MEGA's term reduction step. It refines both QCS and CCS.
Step 2.c.1 refines QCS. For each disjunctive term in the QCS, MEGA examines each positive-labeled instance and uses function f to compute the probability that the disjunctive term should be eliminated. MEGA then calls procedure *Vote* to tally the votes among the positive-labeled instances and compares the vote with threshold K_d to decide whether that disjunctive term is to be removed. If sufficient numbers of positive-labeled instances contradict the QCS with respect to a disjunctive term (i.e., if the threshold is exceeded), the term is removed from the QCS. We discuss procedure *Vote*, which decides how aggressive MEGA is in eliminating terms,

Algorithm MEGA

Input: U, K_c, K_d, K_α;

Output: QCS;

Procedure calls: $f()$, $Vote()$, $Sample()$, $Feedback()$, $Collapsed?()$, $Converged?()$;

Variables: c, d, u, y, $P_{t_i|y_j}$, $P_{t_i|y}$;

Begin

1 Initialize the version space
$QCS \leftarrow \{d_1, d_2, \ldots\}$; $CCS \leftarrow \{c_1, c_2, \ldots\}$;

2 Refine query concept via relevance feedback
While (*not Collapsed*?(QCS, CCS) *and not Converged*?(QCS, CCS))

2.a Select informative samples
$u \leftarrow Sample(QCS, CCS, U, K_\alpha)$;

2.b Solicit user feedback
For each $u_i \in u$
$y_i \leftarrow Feedback(u_i)$;

2.c Reduce irrelevant terms

2.c.1 Refine k-CNF using positive samples
For each $d_i \in QCS$
For each $y_j \in y^+$
$P_{d_i|y_j} \leftarrow f(d_i, O_j, QCS)$;
$P_{d_i|y^+} \leftarrow Vote(y^+, P_{d_i|y_j \in y^+}, \gamma)$;
If $(P_{d_i|y^+} > K_d)$
$QCS \leftarrow QCS - \{d_j\}$;

2.c.2 Refine k-DNF using negative samples
For each $c_i \in CCS$
For each $y_j \in y^-$
$P_{c_i|y_j} \leftarrow f(c_i, O_j, CCS)$;
$P_{c_i|y^-} \leftarrow Vote(y^-, P_{c_i|y_j \in y^-}, \gamma)$;
If $(P_{c_i|y^-} > K_c)$
$CCS \leftarrow CCS - \{c_j\}$;

2.d Bookkeeping
$U \leftarrow U - u$;

3 Return query concept
Output QCS;

End

Figure 2: Algorithm MEGA.

in Section 4.5.

Step 2.c.2 refines CCS. Similar to the steps in 2.c.1, for each conjunctive term in the CCS, MEGA examines each negative-labeled instance, and uses function f to compute the probability that the conjunctive term should be eliminated. MEGA then calls procedure *Vote* to tally the votes among the negative-labeled instances. Then it compares the vote with threshold K_c to decide whether that conjunctive term is to be removed. If sufficient numbers of negative-labeled instances satisfy the CCS with respect to a conjunctive term, the term is removed from the CCS.

Step 2.d: Perform bookkeeping by reducing the unlabeled pool.

The refinement step terminates when the learning process converges to the target concept ($Converged? = true$) or the concept is collapsed ($Collapsed? = true$). ($Converged?$ and $Collapsed?$ are defined below.) In practice, the refinement stops when no unlabeled instance u can be found between the QCS and the CCS.

Definition 3: $Converged?(QCS, CCS)$

$$Converged?(QCS, CCS) \leftarrow true\ if\ CCS == QCS;\quad false\ otherwise.$$

Definition 4: $Collapsed?(QCS, CCS)$

$$Collapsed?(QCS, CCS) \leftarrow true\ if\ CCS \in QCS;\quad false\ otherwise.$$

Remarks

Notice that MEGA does not require seeding a query with relevant images. MEGA generates random images initially to seek for user feedback. If all of the images generated in the first round of its relevance feedback are irrelevant, it will use the negative images to reduce the set of potentially relevant images substantially so that a relevant image will be generated in the next round with high probability.

4.2 Sampling

MEGA invokes procedure *Sample* to select the next K_α unlabeled instances to ask for user feedback. From the college-admission example presented in Section 1, we learn that if we would like to minimize our work (i.e., call a minimum number of friends), we should choose our samples judiciously. But, what constitutes a good sample? We know that we learn nothing from a sample if

- It agrees with the concept in all terms.
- It has the same attributes as another sample.
- It is unlikely to be labeled positive.

To make sure that a sample is useful, MEGA employs two strategies:

1. Bounding the sample space: MEGA avoids choosing useless unlabeled instances by using the CCS and QCS to delimit the sampling boundary.

2. Maximizing the usefulness of a sample: MEGA chooses a sample that shall remove the maximum expected number of disjunctive terms. In other words, MEGA chooses a sample that can maximize the expected generalization of the concept.

It may appear that if we pick a sample that has more dissimilar disjunctions (compared to the QCS), we may have a better chance of eliminating more disjunctive terms. This is, however, not true. A sample must be labeled by the user as positive to be useful for refining QCS. Unfortunately, a sample with more disjunctions that are dissimilar to the target concept is less likely to be labeled positive. Therefore, in choosing a sample, there is a tradeoff between those with more contradictory terms and those more likely to be labeled positive.

Let Ψ denote the number of disjunctions remaining in the concept. The number of disjunctions that can be eliminated in the current round of sampling (denoted as ψ) is between zero and Ψ. We can write the probability of eliminating ψ terms as $P_e(\psi)$. $P_e(\psi)$ is a monotonically decreasing function of ψ.

Our MEGA scheme attempts to find the ψ that can eliminate the maximum expected number of disjunctive terms, given a sample. MEGA's objective function can be written as

$$\psi^* = argmax_\psi E(\psi) = argmax_\psi(\psi \times P_e(\psi)). \tag{1}$$

To solve ψ^*, we must know $P_e(\psi)$, which can be estimated by the two methods below: *probabilistic estimation* and *empirical estimation*.

4.3 Probabilistic Estimation

We first consider how to estimate ψ^* using a probability model. As we have seen in the college-admission example, if a sample contradicts more disjunctive terms, it is more likely to be labeled negative (i.e., less likely to be labeled positive). For example, a sample that contradicts predicate P_1 is labeled negative only if P_1 is in the user's query concept. A sample that contradicts both predicates P_1 and P_2 is labeled negative if either P_1 or P_2 is in the user's query concept.

Formally, let random variable Φ_i be 1 if P_i is in the concept and 0 otherwise. For simplicity, let us assume that the Φ_i's are iid (independent and identically distributed), and the probability of Φ_i being 1 is p $(0 < p < 1)$. For a sample contradicting ψ disjunctive terms, it is marked positive only when none of these ψ terms appears in the user's query concept. This probability is $(1-p)^\psi$. If we substitute $P_e(\psi)$ by $(1-p)^\psi$ on the right-hand side of Equation 1, we get

$$E(\psi) = \psi(1-p)^\psi.$$

If we take the derivative of $E(\psi)$, we can find the optimal ψ value, denoted by ψ^*:

$$\psi^* = \Psi, \ if \ \frac{1}{ln\frac{1}{1-p}} > \Psi, \quad \psi^* = \frac{1}{ln\frac{1}{1-p}}, \ otherwise.$$

Of course, it may be too strong an assumption that the probability p of all disjunctions is iid. However, we do not need a precise estimation here for the following two reasons:

1. Precise estimation may not be feasible and can be computationally intensive.

2. An approximate estimation is sufficient for bootstrapping. Once the system is up and running for a while and collects enough data, it can empirically estimate $P_e(\psi)$ using its past experience. We discuss this process next.

4.4 Empirical estimation

MEGA can estimate $P_e(\psi)$ based on its past experience. For each sample it presents, it records how many disjunctions the sample contradicts with respect to the query concept and whether the sample is labeled positive. Once the system collects a sufficient amount of data, we can estimate $P_e(\psi)$ empirically. We then pick the ψ^* that can eliminate the maximum expected number of disjunctive terms.

Again, a reasonable approach to estimate $P_e(\psi)$ is to use probabilistic estimation when the system first starts and then switches to empirical estimation when the system has collected sufficient data. In such a case there can be an abrupt transition when MEGA switches from one estimation to the other, since the two estimates of $P_e(\psi)$ may differ substantially. This may lead to a sudden change in behavior of the sampling component of the active learner. To remedy this problem, we employ a Bayesian smoothing approach. Essentially the probabilistic estimation is the prior guess at the distribution over ψ and the empirical approach is the guess based purely on the data that MEGA has seen so far. The Bayesian approach combines both of these guesses in a principled manner. Before we start, we imagine that we have seen a number of samples of ψ. After refinement iteration we gather new samples for ψ; then we add them to our current samples and adjust $P_e(\psi)$.

For example, before we start, we assume that we have already seen samples with $\psi = 1$ being labeled positive three out of five times and samples with $\psi = 2$ being labeled positive seven out of 20 times. In other words, we have successfully eliminated $\psi = 1$ term three times out of five, and we have successfully eliminated $\psi = 2$ terms 7 times out of 20. Thus initially $P_e(\psi = 1) = 3/5 = 0.6$ and $P(\psi = 2) = 7/20 = 0.35$. Now suppose we do a query and in which we observe a sample with $\psi = 2$ being labeled positive. Then our new distribution is $P(\psi = 1) = 3/5$ and $P(\psi = 2) = 8/21$. We continue in this manner. At first the prior assumption has quite an effect on our guess about the distribution. The more imaginary samples we have in our prior assumption, the larger its effect. For instance, if we assume that $\psi = 1$ being labeled positive 30 out of 50 times and that $\psi = 2$ being labeled positive 70 out of 200 times, it takes more real samples to change $P_e(\psi)$. With time, the more real samples we get, the less the effect of the prior assumption becomes, until eventually it has virtually no effect, and the observed data dominate the expression. This procedure gives us a smooth transition between the "probabilistic" and the "empirical" methods.

Remarks

Once the criteria for selecting samples is determined, we use Clindex, a clustering/indexing combined indexer to retrieve the most informative samples. We discuss this high-dimensional access

method in detail in [16].

Sampling Example

Let us revisit the college-admission example to explain how MEGA works. Suppose Joe is interested in applying for a graduate school. He knows the profiles of a few students in his senior class, but he does not know if they were admitted to that particular school. According to MEGA, we write the initial CCS and QCS in 2-CNF and 2-DNF, respectively, as

$$QCS = (GRE = high) \wedge (GPA = high) \wedge (Publications = true) \wedge (GRE = high \vee GPA = high)$$
$$\wedge\ (Publications = true \vee GPA = high)\ \wedge\ (GRE = high \vee Publications = true).$$
$$CCS = (GRE = high) \vee (GPA = high) \vee (Publications = true) \vee (GRE = high \wedge GPA = high)$$
$$\vee\ (Publications = true \wedge GPA = high)\ \vee\ (GRE = high \wedge Publications = true).$$

Suppose ψ^* is one. Joe starts by calling his friends whose "profile" fails by exactly one disjunctive term. Joe calls three people and two give him a positive answer, as shown in Table 3.

Round #	*Name*	*GPA*	*GRE*	*Has Publications?*	*Was Admitted?*
1st	*Joe*	*high*	*high*	*false*	*true*
	Emily	*high*	*low*	*true*	*true*
	Dora	*low*	*high*	*true*	*false*
2nd	*Kevin*	*high*	*low*	*false*	*false*

Table 3: MEGA Sampling Rounds.

Based on the feedback, Joe can generalize the concept to $QCS = (GPA = high) \wedge (Publications = true \vee GPA = high)\ \wedge\ (GRE = high \vee Publications = true)\ \wedge\ (GPA = high \vee GRE = high)$. At the same time, the CCS is shrunk to $CCS = (GPA = high)\ \vee\ (GRE = high \wedge GPA = high)\ \vee\ (Publications = true \wedge GPA = high)$.

In the second round, Joe attempts to call friends to see if any of the remaining terms can be removed. He calls Kevin, whose profile is listed in the table. Since this sample is labeled negative, the QCS is not changed. But the CCS is reduced to $(GRE = high \wedge GPA = high) \vee (Publications = true \wedge GPA = high)$.

Simplifying and rewriting both QCS and CCS gives us the following identical expression:

$$QCS = (GPA = high)\ \wedge\ (GRE = high \vee Publications = true).$$

The concept converges and the refinement terminates at this point. We have learned the admission criterion — a high GPA and either a high GRE or publications □

4.5 Procedure Vote

In the previous example, we assume that all samples are noise-free. This assumption may not be realistic. There can be two sources of noise:

- Model bias: The target concept may not be in k-CNF.
- User errors: A user may label some positive instances negative and vice versa.

Procedure Vote
Input: y, $P_{t_i|y_j \in y}$, γ;
Output: $P_{t_i|y}$;
Begin
Sort $P_{t_i|y_j}$ in the descending order;
Return the γth highest $P_{t_i|y_j}$;
End

Figure 3: Procedure Vote.

Procedure *Vote* controls the strictness of voting using γ. The larger the value of γ is, the more strict the voting is and therefore the harder it is to eliminate a term. When the noise level is high, we have less confidence in the correctness of user feedback. Thus, we want to be more cautious about eliminating a term. Being more cautious means increasing γ. Increasing γ, however, makes the learning process converge more slowly. To learn a concept when noise is present, one has to buy accuracy with time. In Section 5.1.3 we show how the voting threshold γ affects search accuracy and convergence speed through our experimental results, and we suggest heuristics for selecting a proper γ.

5 Experiments

In this section we report our experimental results. The goals of our experiments were to evaluate

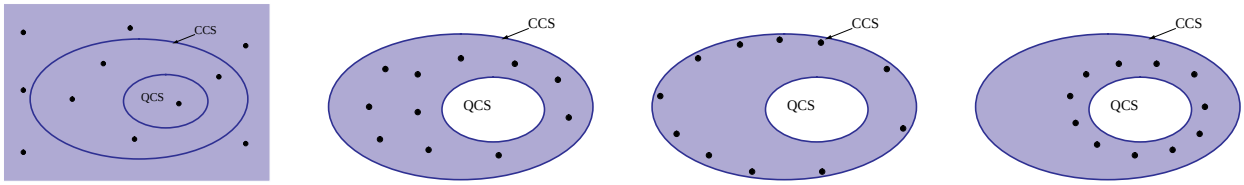
1: Whether MEGA can learn a concept accurately in the presence of a large number of irrelevant attributes.

2: Whether MEGA can converge to a target concept faster than traditional sampling schemes.

3: Whether MEGA is robust for noisy data or under situations in which the unknown target concept is not expressible in the provided hypothesis space.

We assume all target concepts are in 3-CNF. To conduct our experiments, we used both synthesized data and real-world data.

- Synthesized data. We generated three datasets using two different distributions: uniform and Gaussian. Each instance had 10 attributes between 0 and 1. The values of each attribute in a dataset were independently generated. For the Gaussian distribution, we set its mean to 0.5 and its standard deviation to 1/6. Each dataset had $10,000$ vectors.



(a) Random (b) B-Random (c) Aggressive (d) MEGA

Figure 4: Sampling Schemes.

- Real-world data. We conducted experiments on a 2,000-image dataset collected from Corel image CDs and the Internet. The images in the dataset belonged to fifteen categories. Each image was characterized by a 144-dimension attribute vector (described in Section 5.2).

We used *precision* and *recall* to measure performance. We tallied precision/recall for up to only ten iterations, since we deemed it unrealistic to expect an interactive user to conduct more than ten rounds of relevance feedback. We compared MEGA with the four sampling schemes employed by some traditional systems: *random* (used by traditional AI schemes such as decision trees and neural networks), *bounded random* (used by QBC), *nearest neighbor* (used by MARS [12, 19, 20]), *query expansion* (used by [21, 26]). We also examined an *aggressive* sampling strategy that attempts to eliminate maximum number of terms without considering whether a sample might be labeled positive.

Figure 4 shows how some of these sampling algorithms work. The main features of the sampling schemes are given below.

- *Random*: Samples are randomly selected from the bulk of the domain (Figure 4(a)).
- *Bounded Random*: Samples are randomly selected from between QCS and CCS (Figure 4(b)).
- *Nearest Neighbor*: Samples are selected from the nearest neighborhood of the center of the positive-labeled instances.
- *Query Expansion*: Samples are selected from the neighborhood of multiple positive-labeled instances.
- *Aggressive*: Samples are selected from unlabeled ones that satisfy the most general concepts in CCS (Figure 4(c)).
- *MEGA*: Samples are selected from between QCS and CCS to eliminate the maximum expected number of terms (Figure 4(d)).

We ran experiments on datasets of different distributions and repeated each experiment ten times. The experimental results are presented in two groups. We first show the results of the experiments on the synthesized datasets. We then show the results on a 2,000-image databset.

5.1 Query Concept Learning Applied to Synthesized Datasets

We tested many target concepts on the two synthesized datasets. Due to space limitations, we present only three representative test cases, those that represent a disjunctive concept, a conjunctive of disjunctions, and a complex concept with more terms. (Additional results are shown in the extended version of this paper [4].) The three tests are

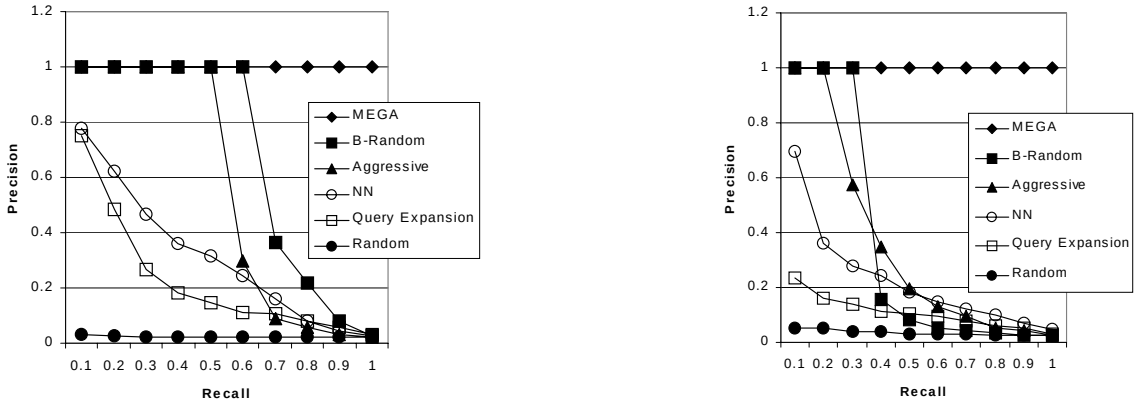
1: $P_1 \vee P_2$,

2: $(P_1 \vee P_2) \wedge P_3$,

3: $P_1 \wedge (P_2 \vee P_3) \wedge (P_4 \vee P_5 \vee P_6) \wedge (P_2 \vee P_4 \vee P_7)$,

We first assume that the dataset is free of user errors and we set the sample size K_α to 20. In the remainder of this section, we first report our initial results, and then we report the effects of model bias (Sections 5.1.2), of user errors (Section 5.1.3), and of sample size K_α (Section 5.1.4) on MEGA.

5.1.1 Experimental Results



(a) $(P_1 \vee P_2) \wedge P_3$ (b) Complex

Figure 5: Precision vs. Recall.

Figure 5 presents the precision/recall after three user iterations of the six sampling schemes learning the two concepts, $(P_1 \vee P_2) \wedge P_3$ and $P_1 \wedge (P_2 \vee P_3) \wedge (P_4 \vee P_5 \vee P_6) \wedge (P_2 \vee P_4 \vee P_7)$. The performance trend of the six schemes is similar at different numbers of iterations. We deem three iterations a critical juncture where a user would be likely to lose his/her patience, and thus we first present the results at the end of the third iteration. The performance curve of MEGA far exceeds that of the other five schemes at all recall levels. Note that for learning both concepts, MEGA achieves 100% precision at all recall levels.

Next, we were interested in how search accuracy improved with respect to the number of user iterations. This improvement trend tells us how fast a scheme can learn a target concept. We present below a set of tables and charts where we fix recall at 0.5 and examine the improvement in precision with respect to the number of iterations.

Rnd #	*Random*	*B-Random*	*N-Neighbor*	*Q-Expansion*	*Aggressive*	*Algorithm MEGA*
1	0.23715	0.23715	0.20319	0.20319	0.23715	0.23715
2	0.44421	0.44421	0.48207	0.44422	0.44421	0.30098
3	0.49507	0.50389	0.41036	0.45219	0.50389	1.00000
4	0.50389	1.00000	0.36753	0.51394	1.00000	1.00000
5	1.00000	1.00000	0.35857	0.78088	1.00000	1.00000
6	1.00000	1.00000	0.33865	0.88247	1.00000	1.00000
7	1.00000	1.00000	0.32669	0.93028	1.00000	1.00000
8	1.00000	1.00000	0.32271	0.93028	1.00000	1.00000
9	1.00000	1.00000	0.29880	0.93028	1.00000	1.00000
10	1.00000	1.00000	0.32570	0.93028	1.00000	1.00000

Table 4: Learning $P_1 \vee P_2$ Applied to a Uniform Dataset.

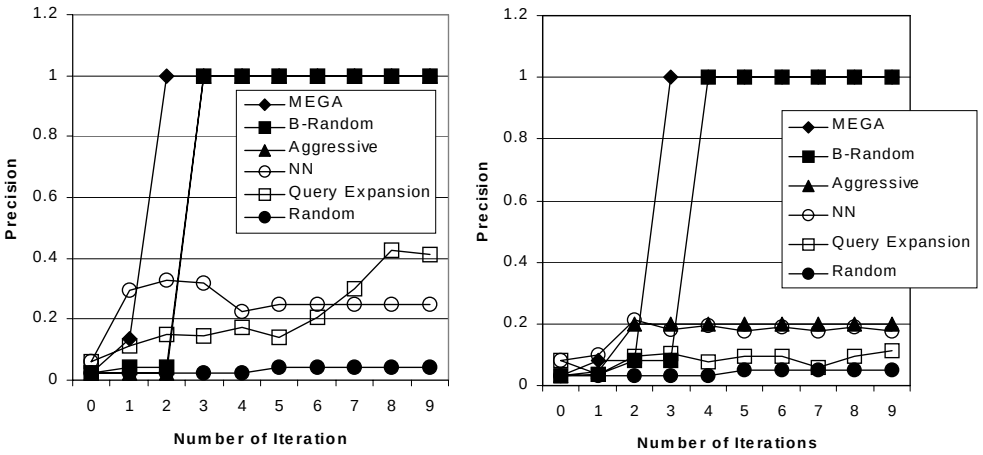
Tables 4 and 5 present the precision of six sampling schemes in learning $P_1 \vee P_2$ in ten rounds of relevance feedback. These tables show that MEGA consistently converges to the target concept in the smallest number of iterations. Applied to the Gaussian dataset, MEGA converges after four iterations. The random sampling scheme requires on average two more iterations to converge. The performance of the bounded random scheme and the performance of the aggressive scheme fall between that of the random scheme and that of MEGA. On the aggressive scheme, which attempts to remove as many terms as possible, the chosen samples are less likely to be labeled positive and hence make less of a contribution to the progress of learning the QCS. We will show shortly that the gaps in performance between MEGA and the other schemes increase as the target concept becomes more complex.

Rnd #	*Random*	*B-Random*	*N-Neighbor*	*Q-Expansion*	*Aggressive*	*Algorithm MEGA*
1	0.08236	0.08236	0.29970	0.29970	0.08236	0.08236
2	0.22178	0.22178	0.65722	0.46684	0.36241	0.32438
3	0.37332	0.37332	0.64907	0.47027	0.80584	0.65982
4	0.38200	0.51249	0.64134	0.46598	0.80584	1.00000
5	0.51249	1.00000	0.63941	0.66237	0.80584	1.00000
6	1.00000	1.00000	0.62782	0.46491	0.80584	1.00000
7	1.00000	1.00000	0.61000	0.47135	0.80584	1.00000
8	1.00000	1.00000	0.61000	0.61258	0.80584	1.00000
9	1.00000	1.00000	0.61000	0.48830	0.80584	1.00000
10	1.00000	1.00000	0.61000	0.64198	0.80584	1.00000

Table 5: Learning $P_1 \vee P_2$ Applied to a Gaussian Dataset.

The results of all datasets and all subsequent tests show that both the nearest neighbor and the query expansion schemes converge very slowly. The result is consistent with that reported in [21, 22], which shows that the query expansion approach does better than the nearest neighbor approach but both suffer from slow convergence. Sampling in the nearest neighborhood tends to result in low precision/recall if the initial query samples are not perfect.

The precision at a given recall achieved by the experiments applied to the Gaussian dataset is lower than that of the experiments applied to the uniform dataset. This occurs because when an initial query point falls outside of, say, two times the standard deviation, we may not find enough positive examples in the unlabeled pool to eliminate all superfluous disjunctions. Since



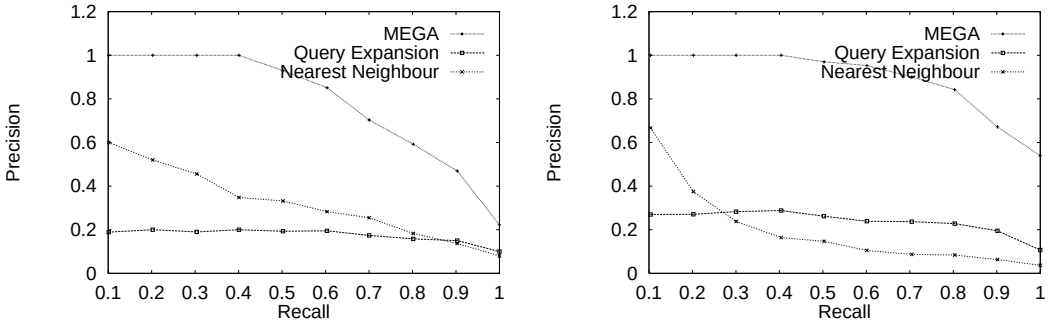
(a) $(P_1 \vee P_2) \wedge P_3$ (b) *Complex Query*

Figure 6: Precision of Six Schemes at Recall = 50%.

this situation is rare, the negative effect on the average precision/recall is insignificant. The performance gaps between the six sampling schemes were similar when we applied them to the two datasets; therefore, we report only the results of the experiments on the uniform dataset in the remainder of this section.

Figure 6 depicts the results of the second and third tests on the uniform dataset. The figure shows that MEGA outperforms the other scheme (in precision at a fixed recall) by much wider margins. It takes MEGA only three iterations to learn these concepts, whereas the other schemes progress more slowly. Schemes like nearest-neighbor and query expansion fail miserably because they suffer from severe model bias. Furthermore, they cannot eliminate irrelevant attributes quickly.

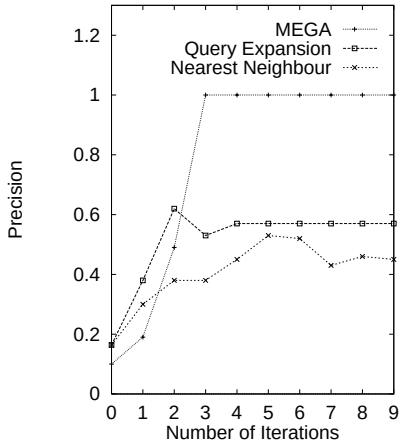
5.1.2 Model Bias Test



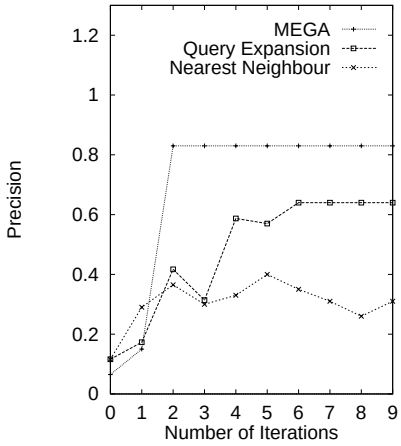
(a) $\alpha = 0.3$ (b) $\alpha = 0.5$

Figure 7: Recall vs. Precision (Model Bias Test).

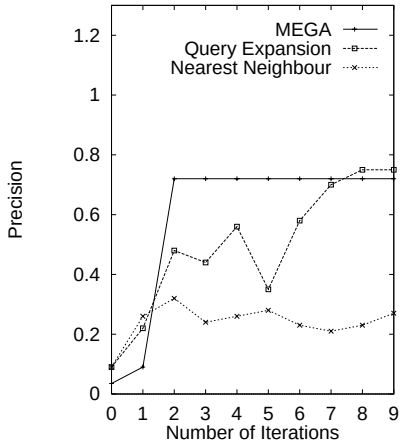
We have shown that MEGA outperforms the other five sampling schemes significantly when the target query concept is in k-CNF. We now present test cases that favor a convex concept, which can be expressed as a linear weighted sum of attributes to examine how MEGA performs. The target concept we tested is in the form of $\alpha P_1 + (1 - \alpha)P_2$, where the value of α is between



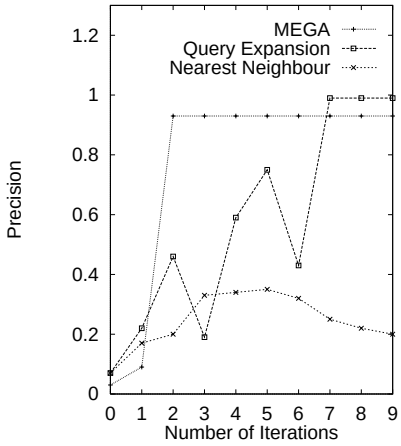
(a) $\alpha = 0(1.0)$



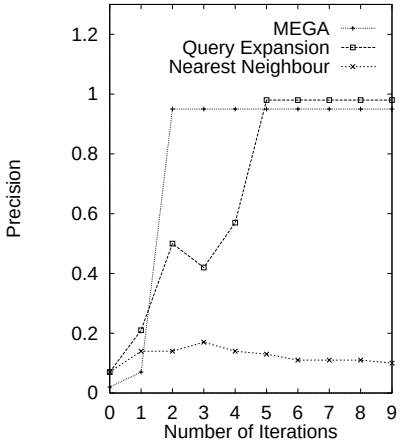
(b) $\alpha = 0.1(0.9)$



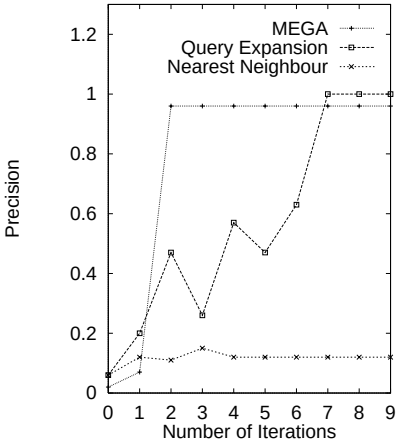
(c) $\alpha = 0.2(0.8)$



(d) $\alpha = 0.3(0.7)$



(e) $\alpha = 0.4(0.6)$



(f) $\alpha = 0.5$

Figure 8: The Effect of Different α's.

zero and one.

In this set of tests, we compared MEGA with the nearest neighbor scheme and the query expansion scheme, which are the representative schemes designed for refining convex concepts. We started by picking 20 random images to see how fast each scheme would converge to the target concepts. Again, we repeated each experiment ten times and recorded each scheme's average precision and recall.

We tested six convex concepts by setting $\alpha = 0, 0.1, \ldots, 0.5$. Below, we report the precision/recall of the three learning methods on two concepts: $0.3P_1 + 0.7P_2$ ($\alpha = 0.3$) and $0.5P_1 + 0.5P_2$ ($\alpha = 0.5$). Setting α in this range makes MEGA suffer from model bias. (We will discuss the reasons shortly.) Figure 7 presents the precision/recall of the three schemes for learning these two concepts after three user iterations. Surprisingly, even though MEGA is not modeled after a convex concept, the performance curve of MEGA far exceeds that of the other two schemes in learning both concepts.

To understand the reasons why MEGA works better than the nearest neighbor and query expansion schemes and how each scheme improves from one iteration to another, we present a set of charts where we fix recall at 0.5 and examine the trend of precision with respect to the number of iterations. (The trend at other recall levels is similar.) Figure 8(a) shows the result of learning concept P_2 (setting $\alpha = 0$). MEGA does very well in this experiment, since it suffers no model bias. Neither the nearest neighbor nor the query expansion scheme does as well because they are slow in refining query concepts and reducing irrelevant attributes.

What if a user does have a weighted linear query-concept in mind? Even so, MEGA can approximate this model fairly well. Figures 8(b), (c), (d), (e), and (f) all show that MEGA achieves higher precision faster than either the nearest neighbor or the query expansion scheme under all α settings. We summarize our observations as follows:

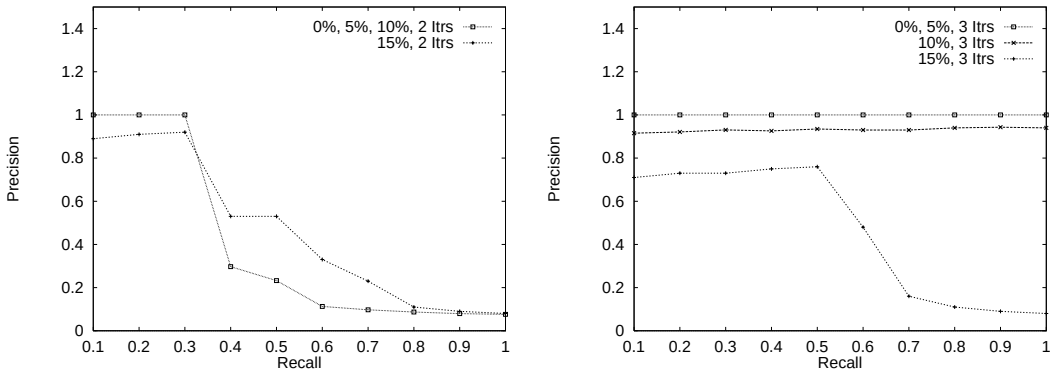
1. When $\alpha = 0$ (or 1), the concept has only one predicate and MEGA has better precision by a wide margin than the two traditional schemes, since it can converge much faster. Even when α is near 0 or 1, the precision of MEGA decreases slightly but still outperforms the traditional schemes, as shown in Figure 8(b). This is because although MEGA suffers slightly from model bias, its fast convergence makes it a better choice when the number of iterations is relatively small.

2. When $\alpha = 0.5$, MEGA can approximate the convex concept by $P_1 \wedge P_2$. Figures 8(e) and (f) show that when α is near 0.5, MEGA trails the query expansion by only a slim margin after five/six user iterations. Although the query expansion scheme eventually converges to the target concept, MEGA's fast improvement in precision in just a couple of iterations makes it more attractive, even though slower learning schemes might eventually achieve a slightly higher precision.

3. Figures 8(c) through (e) show that when α is between 0.2 and 0.4, MEGA suffers from model bias and its achievable precision can be low. However, our primary concern is with the range between three and five iterations that will probably conform to the patience of on-line users. For this purpose, MEGA is more attractive even with its model bias. When

$\alpha = 0.2$, MEGA reaches 70% precision after two iterations whereas the query expansion scheme requires seven iterations to reach the same precision.

5.1.3 User Error Test

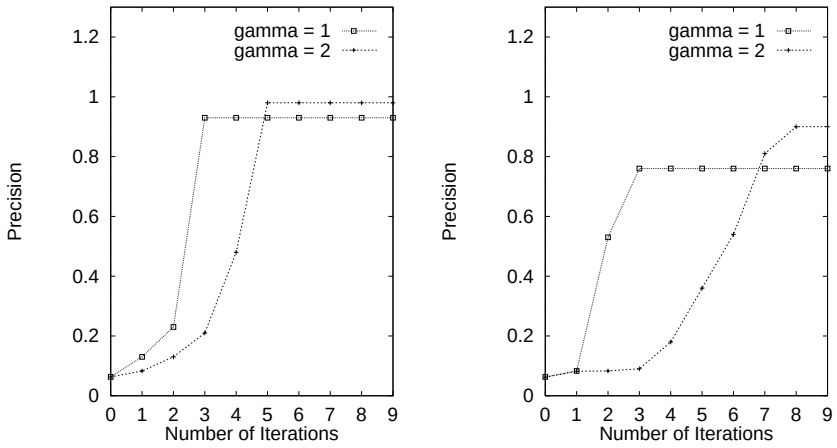


(a) Two Iterations (b) Three Iterations

Figure 9: Precision/Recall Under 0%, 5%, 10% and 15% Noise.

In this experiment MEGA learned the $(P_1 \vee P_2) \wedge (P_3 \vee P_4)$ concept under three different error rates, 5%, 10% and 15%. (A five percent error rate means that one out of twenty samples is mislabeled.) We also used two different γ settings (one and two) to examine the tradeoff between learning speed and accuracy. Figure 9 presents the precision/recall after two or three user iterations under different error rates. MEGA enjoys little to no performance degradation when the noise rates are less than or equal to 10%. When the error rate is 15%, MEGA's search accuracy starts to deteriorate. This experiment shows that MEGA is able to tolerate mild user errors.

Next, we fixed recall at 50% and examined how different error rates and γ settings affect learning precision. Figure 10(a) shows that under both $\gamma = 1$ and $\gamma = 2$ settings, MEGA reaches

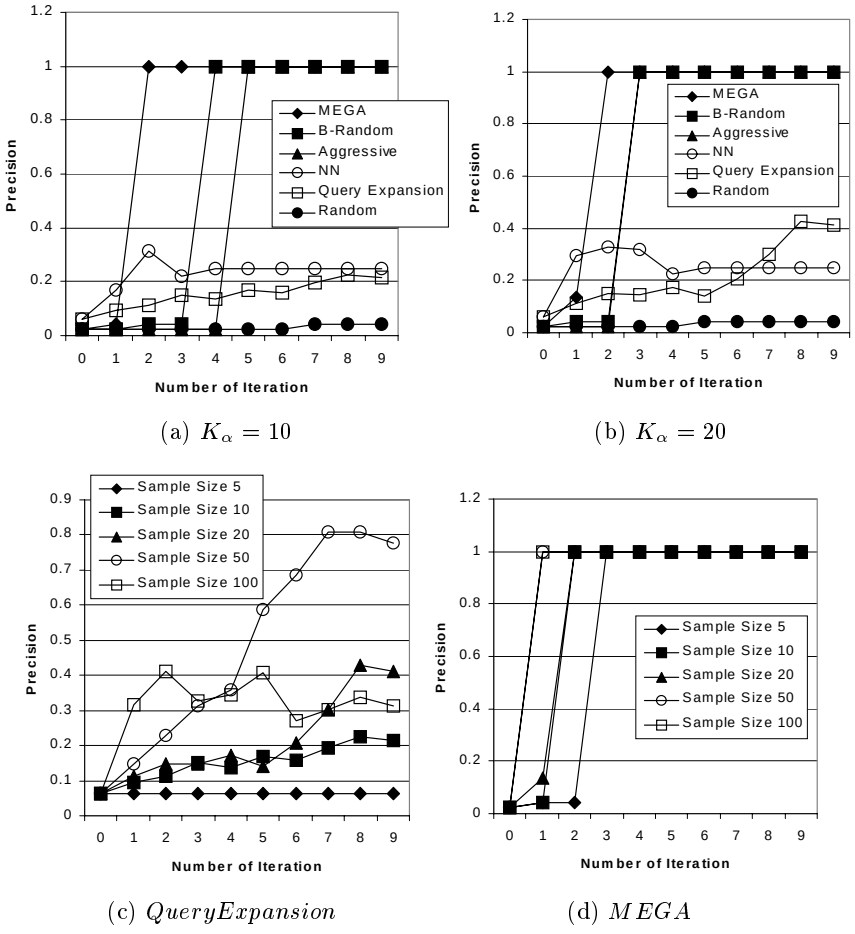


(a) 10% (b) 15%

Figure 10: Effects of Noise.

high precision. However, MEGA's precision improves much faster when $\gamma = 1$ than when $\gamma = 2$. This result does not surprise us, since a lower γ value eliminates terms more aggressively and hence leads to faster convergence. When the noise level is high (15%), Figure 10(b) shows that a low γ setting hinders accurate learning of the target concept. This is because MEGA eliminates terms too aggressively, and the high noise level causes it to eliminate wrong terms. But if we set $\gamma = 2$, we can learn the concept with higher accuracy by slowing down the learning pace. This experiment shows a clear tradeoff between learning accuracy and convergence speed. When the noise level is low, it is preferable to use a less strict voting scheme (i.e., setting a smaller γ) for achieving faster convergence. When the noise level is high, a stricter voting scheme (i.e., using a larger γ) will better maintain high accuracy.

5.1.4 The Effects of Sample Size K_α



(a) $K_\alpha = 10$ (b) $K_\alpha = 20$

(c) *QueryExpansion* (d) *MEGA*

Figure 11: The Effect of Sample Size.

We have so far reported the results of using $K_\alpha = 20$ as the sample size. Intuition tells us that the larger the sample, the faster the learning speed. However, using a large sample may not be realistic. For image retrieval, providing 20 or 30 samples at a time to solicit user feedback is reasonable. For text and music searches, however, the desired sample size tends to be smaller.

We fixed the recall at 50% and listed the precision achieved for learning $(P_1 \vee P_2) \wedge P_3$ by the six schemes under two different sample sizes, 20 and 10, in Figures 11(a) and 11(b). When K_α is cut in half, from 20 to 10, MEGA still converges to the target concept in just two iterations, while other schemes either take more iterations to converge to the target concept, or achieve a lower precision. Figures 11(c) and 11(d) show the effect of the sample size on the convergence rate of the query expansion scheme and MEGA, respectively, for different sample sizes and different numbers of iterations. We can see that the performance of MEGA is relatively less sensitive to the sample size. Even when the sample size is 5, MEGA needs only one more iteration to converge to the target concept. These results reveal that the quality of the samples may be more critical than the quantity of samples.

5.1.5 Observations

We can summarize the above experimental results as follows:

1. Convergence speed: MEGA converges much faster than the other schemes in all cases.
2. Model accuracy: MEGA outperforms the other schemes by a wide margin when the target query concept is in k-CNF. Even when a user's query concept is a weighted linear function, MEGA can approximate it fairly well. The fact that MEGA can achieve a high convergence ratio in a small number of iterations makes it an attractive online learning scheme.
3. Noise tolerance: MEGA does well under noise conditions, including model bias and user errors.

5.2 MEGA Applied to An Image Dataset

We also conducted experiments on a 2,000-image dataset [3]. A 144-dimension attribute vector was extracted for each image containing information about color histograms, color moments, textures, etc [5]. We assumed that query concepts could be modeled in 3-CNF. Each of the query concepts we tested belongs to one of the fifteen image categories: *architecture*, *bears*, *clouds*, *elephants*, *fabrics*, *fireworks*, *flowers*, *food*, *landscape*, *people*, *objectionable images*, *textures*, *tigers*, *tools*, and *waves*. Many categories were added to make the learning task challenging. For instance, three animal categories —*bears*, *elephants*, and *tigers*— all have landscape background which makes the discernment of these categories difficult. *Clouds* and *waves* can appear very similar to each other, as can *flowers* and *fabrics*. Finally, various texture images (e.g., skin, brick, grass, water, etc.) were added to raise the learning difficulty for all categories.

MEGA learned a target concept solely in the attribute space without knowledge about these categories. In each experiment, we began with a set of 20 randomly generated images for querying user feedback. After each iteration, we evaluated the performance by retrieving top-K images based on the concept learned. We recorded the ratio of the images that satisfied the user's concept. We ran each experiment through up to five rounds of relevance feedback, since we deemed it unrealistic to expect an interactive user to conduct numerous rounds of feedback. We ran each experiment ten times with different initial starting samples.

Categories	Iteration 1		Iteration 2		Iteration 3		Iteration 4		Iteration 5	
	Top 10	Top 20	Top 10	Top 20	Top 10	Top 20	Top 10	Top 20	Top 10	Top 20
Architecture	0.470	0.410	0.750	0.560	0.900	0.645	0.960	0.750	0.990	0.865
Bears	0.210	0.165	0.550	0.340	0.750	0.505	0.870	0.630	0.930	0.710
Clouds	0.340	0.210	0.750	0.635	0.990	0.940	0.980	0.980	0.950	0.970
Elephants	0.570	0.375	0.590	0.450	0.720	0.505	0.790	0.630	0.870	0.680
Fabrics	0.410	0.285	0.620	0.440	0.860	0.595	0.900	0.755	0.910	0.855
Fireworks	0.690	0.585	1.000	0.965	1.000	1.000	1.000	0.995	1.000	1.000
Flowers	0.330	0.205	0.750	0.525	0.960	0.720	1.000	0.850	0.990	0.960
Food	0.580	0.465	0.980	0.950	1.000	0.985	1.000	0.995	1.000	1.000
Landscape	0.270	0.200	0.550	0.365	0.730	0.500	0.850	0.615	0.890	0.715
Objectionable	0.420	0.275	0.860	0.705	1.000	0.890	1.000	0.945	1.000	0.985
People	0.360	0.240	0.550	0.350	0.740	0.470	0.810	0.570	0.940	0.695
Textures	0.400	0.300	0.850	0.675	0.970	0.835	1.000	0.945	1.000	0.960
Tigers	0.480	0.345	0.740	0.610	0.910	0.715	0.910	0.765	0.930	0.820
Tools	0.500	0.320	0.940	0.865	1.000	0.990	1.000	0.995	1.000	0.995
Waves	0.400	0.340	0.630	0.445	0.880	0.735	0.970	0.855	0.980	0.930
Avarage	0.429	0.315	0.741	0.592	0.894	0.735	0.936	0.818	0.959	0.876

Table 6: Experimental Results on Image Dataset.

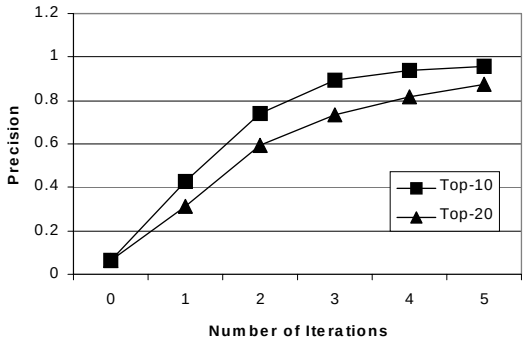


Figure 12: Average Precision of Top-10 and Top-20 Queries.

Table 6 shows the precision of the ten query concepts—for $K = 10$ or 20. (Recall is not presented in this case because it is irrelevant.) For each of the queries, after three iterations the results were satisfactory concerning the quality of the top-10 retrieval. For top-20 retrieval, only one more iteration was required to surpass 81% precision. Finally, Figure 12 shows the average precision of the top-10 and top-20 retrieval of all queries with respect to the number of iterations.

6 Conclusion

We have proposed MEGA, a query-concept learner that learns query concepts quickly and with a small number of labeled instances. Instead of returning the "best" results after each round of relevance feedback as traditional methods do, MEGA generates intelligent samples to maximize the collection of relevant information for trimming the candidate concept-space. We have shown that the sample selection and concept refinement steps of MEGA work in a complementary manner

to achieve fast convergence to query concepts.

Our experimental results showed that MEGA outperforms traditional sampling schemes in all query scenarios when the number of labeled instances is limited. In addition, we showed that MEGA can cope well with model bias and user errors. We have implemented a Perception-Based Image Retrieval (PBIR) system, which employs MEGA as its query-concept learner [3]. Through MEGA, the PBIR system can fuzzily capture a query concept, usually in two to three feedback iterations, and can comprehend a target concept very well in three to five iterations.

We believe an intelligent concept-learner like MEGA is needed for easier and faster information access. Because it is more personal and user-friendly, such a tool may open access to underprivileged users, who may not yet enjoy the full benefit of the information revolution.

References

[1] L. Breiman. Bagging predicators. *Machine Learning*, 26:123–140, 1996.

[2] L. Breiman. Arcing classifiers. *The Annals of Statistics*, 26:801–849, 1998.

[3] E. Chang and T. Cheng. Perception-based image retrieval. *ACM Sigmod (Demo)*, May 2001.

[4] E. Chang and B. Li. Mega — the maximizing expected generalization algorithm for learning complex query concepts (extended version). *Technical Report http://www-db.stanford.edu/∼echang/mega.ps*, November 2000.

[5] E. Chang, B. Li, and C. Li. Towards perception-based image retrieval. *IEEE Content-Based Access of Image and Video Libraries*, pages 101–105, June 2000.

[6] C. Chen. *Fuzzy Logic and Neural Network Handbook*. McGraw-Hill, 1996.

[7] I. J. Cox, M. L. Miller, T. P. Minka, T. V. Papathomas, and P. N. Yianilos. The bayesian image retrieval system, pichunter: Theory, implementation and psychological experiments. *IEEE Transaction on Image Processing (to appear)*, 2000.

[8] R. Fagin. Fuzzy queries in multimedia database systems. *ACM Sigacr-Sigmod-Sigart Symposium on Principles of Database Systems*, 1998.

[9] R. Fagin and E. L. Wimmers. A formula for incorporating weights into scoring rules. *International Conference on Database Theory*, pages 247–261, 1997.

[10] Y. Freund, H. S. Seung, E. Shamir, and N. Tishby. Selective sampling using the query by committee algorithm. *Machine Learning*, 28:133–168, 1997.

[11] K. Fukanaga. *Introduction to Statistical Pattern Recognition*. Boston: Academic Press, 1990.

[12] Y. Ishikawa, R. Subramanya, and C. Faloutsos. Mindreader: Querying databases through multiple examples. *VLDB*, 1998.

[13] K. S. Jones and P. W. (Editors). *Readings in Information Retrieval*. Morgan Kaufman, July 1997.

[14] M. Kearns, M. Li, and L. Valiant. Learning boolean formulae. *Journal of ACM*, 41(6):1298–1328, 1994.

[15] M. Kearns and U. Vazirani. *An Introduction to Computational Learning Theory*. MIT Press, 1994.

[16] C. Li, E. Chang, H. Garcia-Molina, and G. Wiederhold. Clustering for approximate similarity queries in high-dimensional spaces. *IEEE Transaction on Knowledge and Data Engineering (to appear)*, 2001.

[17] T. Michell. *Machine Learning*. McGraw Hill, 1997.

[18] T. M. Mitchell. *Machine Learning*. McGraw-Hill, 1997.

[19] M. Ortega, Y. Rui, K. Chakrabarti, A. Warshavsky, S. Mehrotra, and T. S. Huang. Supporting ranked boolean similarity queries in mars. *IEEE Transaction on Knowledge and Data Engineering*, 10(6):905–925, December 1999.

[20] K. Porkaew, K. Chakrabarti, and S. Mehrotra. Query refinement for multimedia similarity retrieval in mars. *Proceedings of ACM Multimedia*, November 1999.

[21] K. Porkaew, S. Mehrota, and M. Ortega. Query reformulation for content based multimedia retrieval in mars. *ICMCS*, pages 747–751, 1999.

[22] Y. Rui, T. S. Huang, M. Ortega, and S. Mehrotra. Relevance feedback: A power tool in interactive content-based image retrieval. *IEEE Tran on Circuits and Systems for Video Technology*, 8(5), Sept 1998.

[23] R. Schapire, Y. Freund, P. Bartlett, and W. Lee. Boosting the margin: A new explanation for the effectiveness of voting methods. In *Proceeding of the Fourteenth International Conference on Machine Learning*. Morgan Kaufmann, 1997.

[24] L. Valiant. A theory of learnable. *Proceedings of the Sixteenth Annual ACM Symposium on Theory of Computing*, pages 436–445, 1984.

[25] V. Vapnik. *Statistical Learning Theory*. Wiley, 1998.

[26] L. Wu, C. Faloutsos, K. Sycara, and T. R. Payne. Falcon: Feedback adaptive loop for content-based retrieval. *The* 26th *VLDB Conference*, September 2000.

[27] L. A. Zadeh. Fuzzy sets. *Information and Control*, pages 338–353, 1965.